Proiect Învățare Automată – 2026–2027

Peancă Teodora

1. Alegerea temei (S2)

Titlul proiectului

Analiza și predicția comportamentului clienților într-un magazin online folosind tehnici de învățare automată

Link la dataset și scurtă descriere

Dataset: eCommerce behavior data from multi category store

https://www.kaggle.com/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store

Datasetul conține date despre comportamentul utilizatorilor unui magazin online multi-categorie. Versiunea utilizată conține date pentru octombrie și noiembrie 2019. Fiecare rând reprezintă un eveniment realizat de un utilizator asupra unui produs.

Număr de exemple: 109.950.743

Număr de caracteristici: 9

Caracteristici:

event_time – data și ora evenimentului;

event_type – tipul acțiunii (view, cart, remove_from_cart, purchase);

product_id – identificatorul produsului;

category_id – identificatorul categoriei;

category_code – codul categoriei;

brand – brandul produsului;

price – prețul produsului;

user_id – identificatorul utilizatorului;

user_session – identificatorul sesiunii utilizatorului.

Scurtă descriere a task-ului

Scopul proiectului este realizarea unui model de clasificare care să estimeze dacă interacțiunile unui utilizator cu un produs vor conduce sau nu la o achiziție.

2. Exploratory Data Analysis (S4)

Pentru analiza exploratorie am utilizat un eșantion de 100.000 de exemple, extras din datele pentru octombrie și noiembrie 2019. Datasetul conține 9 caracteristici și informații despre acțiunile utilizatorilor într-un magazin online.

Am folosit Python și biblioteca Pandas pentru a examina datele, a verifica valorile lipsă și a analiza distribuția tipurilor de evenimente.

Analiza valorilor lipsă

Am verificat dacă există valori lipsă în cele 100.000 de exemple analizate. Am observat că două caracteristici conțin valori lipsă:

- category_code –31.974 valori lipsă (31,97%)
- brand – 14.036 valori lipsă (14,04%)

Celelalte caracteristici nu au valori lipsă în eșantionul verificat.

Distribuția tipurilor de evenimente

Am analizat tipurile de evenimente din eșantionul de 100.000 de exemple. Rezultatele obținute sunt:

- view – 95.045 evenimente (95,05%);
- cart – 3.372 evenimente (3,37%);
- purchase – 1.583 evenimente (1,58%).

Se observă că majoritatea interacțiunilor sunt vizualizări de produse, în timp ce adăugările în coș și achizițiile sunt mult mai puține.

Estimarea rezultatelor pe baza unui eșantion mai mic

Pentru a verifica cât de precise pot fi estimările realizate pe un eșantion mai mic, am selectat aleatoriu 10.000 de înregistrări din cele 100.000 analizate. Am calculat numărul de vizualizări, adăugări în coș și achiziții, apoi am înmulțit rezultatele cu 10 pentru a estima distribuția evenimentelor în întregul eșantion de 100.000 de înregistrări.

Am comparat valorile estimate cu cele reale și am calculat eroarea procentuală pentru fiecare tip de eveniment.

Tip eveniment	Estimare (10.000)	Real (100.000)	Eroare absolută	Eroare (%)
view	95.320	95.045	275	0,29%
cart	3.210	3.372	162	4,80%
purchase	1.470	1.583	113	7,14%

Interpretarea rezultatelor

Am comparat estimările obținute din 10.000 de înregistrări cu valorile reale din eșantionul de 100.000 de înregistrări.

Rezultatele arată că eroarea pentru vizualizări este de 0,29%, pentru adăugările în coș de 4,80%, iar pentru achiziții de 7,14%.

Observ că estimarea este foarte apropiată de valoarea reală în cazul vizualizărilor. Diferențele sunt mai mari pentru evenimentele mai puțin frecvente, precum adăugările în coș și achizițiile.

Această analiză arată că putem obține estimări utile și dintr-un eșantion mai mic, dar precizia poate varia în funcție de frecvența evenimentelor.

Notebook Kaggle – codul și rezultatele analizei

Analiza exploratorie a datelor a fost realizată în Kaggle, folosind Python. Notebook-ul conține codul utilizat, rezultatele obținute și reprezentările grafice.

Link către notebook:
https://www.kaggle.com/code/teodorasarahpeanc/ecommerce-eda-invatare-automata

1. Încărcarea bibliotecilor și identificarea fișierelor

Pentru început, am importat bibliotecile necesare pentru analiza datelor și am verificat ce fișiere sunt disponibile în datasetul Kaggle.

In [1]:
```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create a version using "Save & Run All"
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session

# Use the kagglehub client library to attach Kaggle resources like competitions, datasets, and models to your session
# Learn more about kagglehub: https://github.com/Kaggle/kagglehub/blob/main/README.md

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')
```

```
/kaggle/input/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store/2019-Nov.csv
/kaggle/input/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store/2019-Oct.csv
```

2. Citirea și explorarea inițială a datelor

Am încărcat primele 100.000 de înregistrări din fișierul pentru octombrie 2019, pentru a vedea cum sunt organizate datele. Am verificat numărul de exemple, numărul de caracteristici și primele rânduri din dataset.

In [2]:
```
import pandas as pd

fisier = "/kaggle/input/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store/2019-Oct.csv"

df = pd.read_csv(fisier, nrows=100000)

print("Număr de exemple:", len(df))
print("Număr de caracteristici:", len(df.columns))

df.head()
```

```
Număr de exemple: 100000
Număr de caracteristici: 9
```

Out[2]:

	event_time	event_type	product_id	category_id	category_code	brand	price	user_id
0	2019-10-01 00:00:00 UTC	view	44600062	2103807459595387724	NaN	shiseido	35.79	54131214
1	2019-10-01 00:00:00 UTC	view	3900821	2053013552326770905	appliances.environment.water_heater	aqua	33.20	55474871
2	2019-10-01 00:00:01 UTC	view	17200506	2053013559792632471	furniture.living_room.sofa	NaN	543.10	51910725
3	2019-10-01 00:00:01 UTC	view	1307067	2053013558920217191	computers.notebook	lenovo	251.74	55005085
4	2019-10-01 00:00:04 UTC	view	1004237	2053013555631882655	electronics.smartphone	apple	1081.98	53587121

3. Verificarea structurii datelor și a valorilor lipsă

Am verificat tipurile de date ale celor 9 caracteristici și dacă există valori lipsă. Această analiză mă ajută să înțeleg mai bine datasetul și să identific eventualele probleme care trebuie rezolvate înainte de antrenarea modelelor de învățare automată.

In [3]:
```
print("Informații despre dataset:")
df.info()

print("\nValori lipsă pentru fiecare caracteristică:")
print(df.isnull().sum())
```

```
Informații despre dataset:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
 #   Column         Non-Null Count   Dtype  
---  ------         --------------   -----  
 0   event_time     100000 non-null  object 
 1   event_type     100000 non-null  object 
 2   product_id     100000 non-null  int64  
 3   category_id    100000 non-null  int64  
 4   category_code  67413 non-null   object 
 5   brand          85607 non-null   object 
 6   price          100000 non-null  float64
 7   user_id        100000 non-null  int64  
 8   user_session   100000 non-null  object 
dtypes: float64(1), int64(3), object(5)
memory usage: 6.9+ MB

Valori lipsă pentru fiecare caracteristică:
event_time          0
event_type          0
product_id          0
category_id         0
category_code    32587
brand            14393
price                0
user_id              0
user_session         0
dtype: int64
```

4. Analiza tipurilor de evenimente

Am analizat câte evenimente de fiecare tip există în cele 100.000 de înregistrări din octombrie 2019. Am calculat atât numărul de evenimente, cât și procentul fiecăruia, pentru a observa cât de des utilizatorii vizualizează produse, le adaugă în coș sau le cumpără.

In [4]:
```
print("Distribuția tipurilor de evenimente:")
print(df["event_type"].value_counts())

print("\nProcente:")
print(df["event_type"].value_counts(normalize=True) * 100)
```

```
Distribuția tipurilor de evenimente:
event_type
view        97130
purchase     1655
cart         1215
Name: count, dtype: int64

Procente:
event_type
view        97.130
purchase     1.655
cart         1.215
Name: proportion, dtype: float64
```

5. Reprezentarea grafică a tipurilor de evenimente

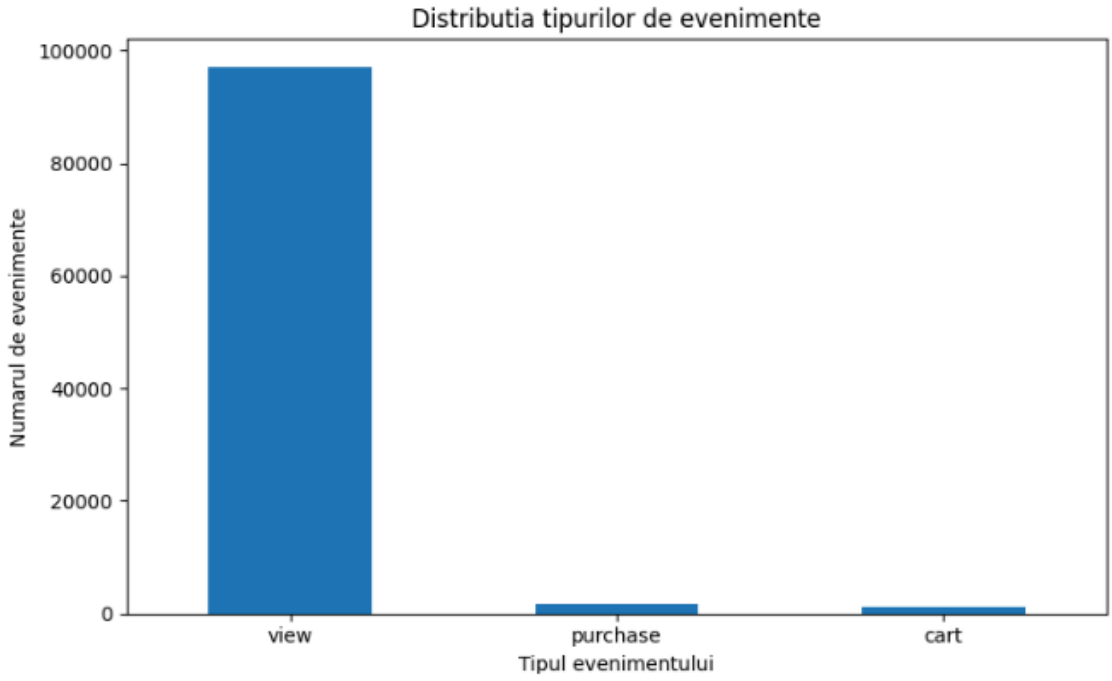
Am realizat un grafic cu bare pentru a vizualiza distribuția tipurilor de evenimente din cele 100.000 de înregistrări analizate. Graficul mă ajută să observ mai ușor diferențele dintre numărul de vizualizări, adăugări în coș și achiziții.

In [5]:

```
import matplotlib.pyplot as plt

df["event_type"].value_counts().plot(
    kind="bar",
    figsize=(8, 5)
)

plt.title("Distributia tipurilor de evenimente")
plt.xlabel("Tipul evenimentului")
plt.ylabel("Numarul de evenimente")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



Crearea eșantionului de date

Am selectat câte 50.000 de înregistrări din octombrie și noiembrie 2019, obținând un eșantion final de 100.000 de exemple pentru analiză.

6. Crearea unui eșantion din octombrie și noiembrie 2019

Deoarece datasetul original este foarte mare, am selectat câte 50.000 de înregistrări din fiecare lună, folosind citirea datelor pe bucăți. Am combinat apoi cele două eșantioane pentru a obține un set de 100.000 de exemple, pe care îl voi utiliza în continuarea analizei.

[6]:
```
import pandas as pd
import numpy as np

baza = "/kaggle/input/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store/"

def esantion_luna(fisier, n=50000):
    bucati = []
    generator = np.random.default_rng(42)

    for chunk in pd.read_csv(baza + fisier, chunksize=500000):
        selectie = generator.random(len(chunk)) < 0.002
        bucati.append(chunk.loc[selectie])

    esantion = pd.concat(bucati, ignore_index=True)
    return esantion.sample(n=n, random_state=42)

octombrie = esantion_luna("2019-Oct.csv")
noiembrie = esantion_luna("2019-Nov.csv")

df = pd.concat([octombrie, noiembrie], ignore_index=True)

print("Număr total de exemple:", len(df))
print("Număr de caracteristici:", len(df.columns))
print(df["event_type"].value_counts())
```

```
Număr total de exemple: 100000
Număr de caracteristici: 9
event_type
view        95045
cart         3372
purchase     1583
Name: count, dtype: int64
```

Distribuția tipurilor de evenimente

Graficul prezintă distribuția acțiunilor utilizatorilor. Se observă că vizualizările produselor sunt mult mai frecvente decât adăugările în coș și achizițiile.

7. Reprezentarea grafică a evenimentelor din cele două luni

Am realizat un grafic cu bare pentru a analiza distribuția evenimentelor din eșantionul combinat de 100.000 de înregistrări. Graficul arată diferențele dintre numărul de vizualizări, adăugări în coș și achiziții din datele selectate pentru octombrie și noiembrie 2019.

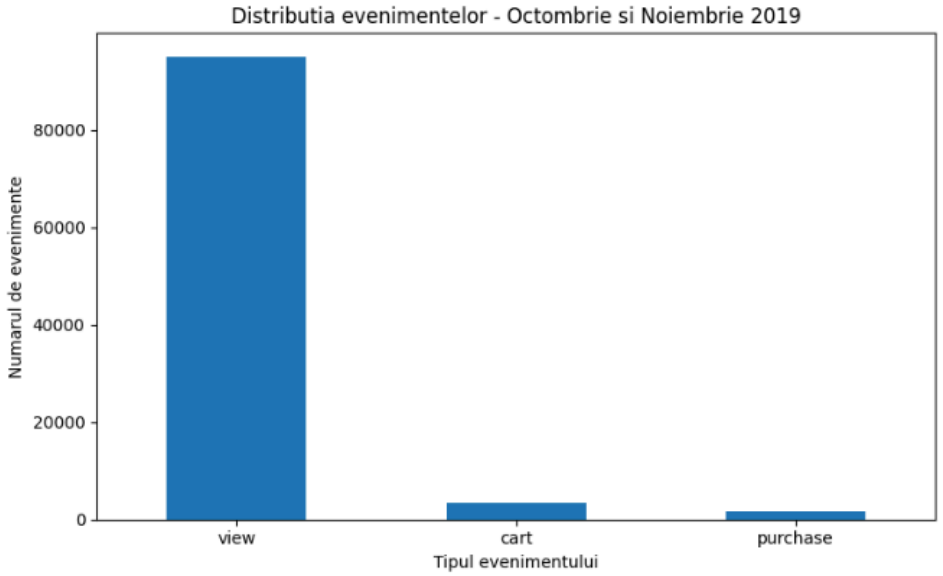
In [7]:
```
import matplotlib.pyplot as plt

df["event_type"].value_counts().plot(
    kind="bar",
    figsize=(8, 5)
)

plt.title("Distributia evenimentelor - Octombrie si Noiembrie 2019")
plt.xlabel("Tipul evenimentului")
plt.ylabel("Numarul de evenimente")
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig("/kaggle/working/distributia_evenimentelor.png", dpi=300, bbox_inches="tight")
plt.show()
```



```
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig("/kaggle/working/distributia_evenimentelor.png", dpi=300, bbox_inches="tight")
plt.show()
```



Verificarea valorilor lipsă

Am verificat valorile lipsă din eșantionul final. Acestea apar doar pentru caracteristicile `category_code` și `brand`.

8. Verificarea valorilor lipsă în eșantionul combinat

Am verificat valorile lipsă din eșantionul de 100.000 de înregistrări selectate din octombrie și noiembrie 2019. Această verificare mă ajută să identific caracteristicile care conțin date incomplete și care ar putea necesita prelucrare în etapele următoare ale proiectului.

In [8]:
```
print(df.isnull().sum())
```

```
event_time          0
event_type          0
product_id          0
category_id         0
category_code    31974
brand            14036
price               0
user_id             0
user_session        0
dtype: int64
```

Estimarea rezultatelor folosind un eșantion mai mic

Am utilizat 10.000 de înregistrări selectate aleatoriu pentru a estima distribuția evenimentelor din eșantionul de 100.000. Am comparat estimările cu valorile reale și am calculat erorile procentuale.

9. Estimarea rezultatelor folosind un eșantion mai mic

În această etapă voi selecta aleatoriu 10.000 de înregistrări din eșantionul de 100.000. Voi calcula distribuția tipurilor de evenimente și voi folosi rezultatele pentru a estima distribuția în cele 100.000 de înregistrări.

Apoi voi compara estimările cu valorile reale și voi calcula erorile, pentru a observa cât de apropiate sunt rezultatele obținute din eșantionul mai mic.

In [9]:

```
# Selectăm aleatoriu 10.000 de exemple
df_mic = df.sample(n=10000, random_state=42)

# Numărăm evenimentele din eșantionul mic
numar_mic = df_mic["event_type"].value_counts()

# Estimăm numărul evenimentelor pentru 100.000 de exemple
estimare = numar_mic * 10

# Calculăm valorile reale
real = df["event_type"].value_counts()

# Comparăm rezultatele
comparatie = pd.DataFrame({
    "Estimare (din 10.000)": estimare,
    "Real (100.000)": real
}).fillna(0)

comparatie["Eroare absoluta"] = abs(
    comparatie["Estimare (din 10.000)"] -
    comparatie["Real (100.000)"]
)
```

```python
# Comparăm rezultatele
comparatie = pd.DataFrame({
    "Estimare (din 10.000)": estimare,
    "Real (100.000)": real
}).fillna(0)

comparatie["Eroare absoluta"] = abs(
    comparatie["Estimare (din 10.000)"] -
    comparatie["Real (100.000)"]
)

comparatie["Eroare (%)"] = (
    comparatie["Eroare absoluta"] /
    comparatie["Real (100.000)"] * 100
).round(2)

print(comparatie)
```

```
            Estimare (din 10.000)  Real (100.000)  Eroare absoluta  Eroare (%)
event_type
view                        95320           95045              275        0.29
cart                         3210            3372              162        4.80
purchase                     1470            1583              113        7.14
```

Interpretarea rezultatelor

Am folosit un eșantion aleatoriu de 10.000 de înregistrări pentru a estima distribuția evenimentelor în setul de 100.000 de exemple.

Am comparat estimările cu valorile reale și am calculat erorile procentuale. Rezultatele arată că eroarea pentru vizualizări este de 0,29%, pentru adăugările în coș de 4,80%, iar pentru achiziții de 7,14%.

Observ că estimarea este foarte apropiată de valoarea reală în cazul vizualizărilor. Diferențele sunt mai mari pentru evenimentele mai puțin frecvente, precum adăugările în coș și achizițiile.

Această analiză arată că putem obține estimări utile dintr-un eșantion mai mic, dar precizia poate varia în funcție de frecvența evenimentelor.

3. Primul model de Machine Learning (S6)

3.1. Scopul modelului

În această etapă am realizat primul model de Machine Learning pentru proiectul meu, folosind datele despre comportamentul clienților dintr-un magazin online.

Am ales să folosesc algoritmul Logistic Regression, pentru a încerca să prezic dacă o sesiune de navigare a unui utilizator se va încheia cu o achiziție.

Pentru antrenarea și testarea modelului am pregătit un set de 47.646 de exemple, obținute din datele analizate anterior. Am folosit câteva caracteristici, precum ora și ziua săptămânii în care a început sesiunea, precum și informații despre primul produs vizualizat.

Scopul acestui prim model este să obțin un rezultat de referință, pe care să îl pot compara ulterior cu modelele îmbunătățite din etapele următoare.

3.2. Pregătirea datelor și antrenarea modelului

Pentru realizarea primului model am folosit datele prelucrate în notebook-ul Kaggle. Am organizat informațiile la nivel de sesiune, astfel încât fiecare exemplu să reprezinte o sesiune de navigare a unui utilizator.

Am urmărit să prezic dacă într-o sesiune are loc cel puțin o achiziție. Pentru aceasta, am creat o variabilă țintă cu două valori: 0 pentru sesiunile fără achiziție și 1 pentru sesiunile în care există o achiziție.

Setul de date a fost împărțit în două părți:

- 80% pentru antrenare – 38.116 exemple;
- 20% pentru testare – 9.530 de exemple.

Am ales algoritmul Logistic Regression, deoarece este potrivit pentru o primă problemă de clasificare binară și oferă un punct de plecare pentru compararea rezultatelor.

La antrenare am utilizat și parametrul `class_weight="balanced"`, pentru a ține cont de faptul că sesiunile cu achiziții sunt mai puțin numeroase decât cele fără achiziții.

3.3. Evaluarea modelului

După antrenare, am testat modelul pe cele 9.530 de exemple rezervate pentru testare. Pentru a evalua performanța, am calculat patru indicatori: Accuracy, Precision, Recall și F1-score.

Rezultatele obținute au fost:

- Accuracy: 40,41%
- Precision: 6,81%

- Recall: 64,77%
- F1-score: 12,32%

Accuracy reprezintă procentul total de predicții corecte, iar Precision arată câte dintre sesiunile prezise ca având o achiziție au avut într-adevăr una. Recall indică proporția sesiunilor cu achiziții pe care modelul a reușit să le identifice, iar F1-score combină Precision și Recall într-o singură valoare.

Observ că modelul identifică aproximativ 65% dintre sesiunile în care există o achiziție, însă face și multe predicții pozitive greșite. Acest lucru explică valoarea redusă a Precision.

Consider că rezultatele sunt modeste, dar utile pentru un prim model. În etapa următoare voi încerca să îmbunătățesc performanța folosind mai multe caracteristici și comparând algoritmi diferiți.

3.4. Înregistrarea experimentului în Weights & Biases

Pentru a păstra rezultatele primului model, am folosit platforma Weights & Biases (W&B), care permite înregistrarea și compararea experimentelor de Machine Learning.

Am creat experimentul „S6-Primul-Model”, în care am salvat valorile obținute pentru Accuracy, Precision, Recall și F1-score.

Experimentul este disponibil la următorul link:

https://wandb.ai/teodorapeanca-university-of-craiova/ECommerce-Customer-Behavior-ML/runs/8js4w5y8

Am folosit această platformă pentru a putea urmări evoluția performanței modelului și pentru a compara rezultatele cu cele pe care le voi obține în etapele următoare.

3.5. Linkuri către proiect

Codul și analiza realizată pentru acest proiect sunt disponibile pe următoarele platforme:

GitHub – repository-ul proiectului:

https://github.com/teodorapeanca/ECommerce-Customer-Behavior-ML

Kaggle – notebook-ul cu analiza datelor și modelul de Machine Learning:

https://www.kaggle.com/code/teodorasarahpeanc/ecommerce-eda-invatare-automata

Weights & Biases – experimentul S6:

https://wandb.ai/teodorapeanca-university-of-craiova/ECommerce-Customer-Behavior-ML/runs/8js4w5y8

Aceste platforme au fost folosite pentru dezvoltarea proiectului, păstrarea codului și urmărirea rezultatelor experimentelor.